KISS Sorcar: A Relentless, Self-Continuing AI Agent for Integrated Software Development

Koushik Sen
Department of Electrical Engineering and Computer Sciences
University of California, Berkeley
Berkeley, CA 94720
ksen@berkeley.edu

Abstract

We present KISS Sorcar, an AI-powered general assistant and integrated development environment built around a layered agent architecture. At its foundation, the KISS Agent implements a budget-aware ReAct loop with native function calling. The Relentless Agent extends this with automatic multi-session continuation and stall detection, enabling arbitrarily long tasks that survive context-window exhaustion. The Sorcar Agent adds coding tools, browser automation, and parallel sub-agent execution. The Stateful Sorcar Agent introduces persistent multi-turn chat sessions, and the Worktree Sorcar Agent isolates every task on a dedicated git branch via worktrees so the developer's main working tree is never modified. We describe the design of each layer, the structured system prompt that governs agent behavior, and the engineering decisions that make the system practical for real-world software development.

1 Introduction

Large language models (LLMs) have demonstrated remarkable ability to generate code, reason about software architecture, and operate developer tools [Chen et al., 2021, Rozière et al., 2023]. However, using a raw LLM as a software engineering assistant exposes several practical gaps: context windows are finite, a single mistake can derail an entire session, and generated changes are difficult to review or revert once applied to a live codebase.

KISS Sorcar addresses these gaps through a five-layer agent hierarchy in which each layer adds exactly one concern:

1. **KISS Agent** — budget-tracked ReAct loop with native function calling.
2. **Relentless Agent** — automatic continuation across sub-sessions with stall detection.
3. **Sorcar Agent** — coding tools, browser automation, and parallel sub-agent execution.
4. **Stateful Sorcar Agent** — persistent multi-turn chat sessions with history recall.
5. **Worktree Sorcar Agent** — git-worktree isolation so every task runs on its own branch.

The name "KISS" reflects the Keep It Simple, Stupid design philosophy: each layer is small, each concern is isolated, and the overall system avoids unnecessary abstraction. The name "Sorcar" pays homage to P. C. Sorcar, the legendary Indian magician, evoking the idea of an agent that performs feats that appear magical yet are grounded in disciplined engineering.

The system is complemented by a carefully structured system prompt that encodes invariants about code quality, verification discipline, and self-improvement. Together, the architecture and the prompt form a practical AI IDE that integrates with Visual Studio Code and operates on real-world repositories.

Preprint.

2 Motivation and Key Design Principles

Finite context, infinite tasks. Even the largest context windows are insufficient for complex refactoring or multi-file bug hunts. Rather than hoping the model finishes before hitting the limit, the system is designed from the ground up to *continue* across multiple sub-sessions, each with its own context window, while preserving a compressed summary of prior progress.

Budget accountability. Every API call costs money. The agent tracks input tokens, output tokens, cache hits, and cache writes at every step, computes a running dollar cost, and enforces per-task and global budget ceilings. If the budget is exceeded, the agent raises a hard error rather than silently accumulating charges.

Safe-by-default code changes. Applying AI-generated changes directly to a developer's working tree is risky: a bad edit can break a build, and reverting requires manual git archaeology. The worktree layer eliminates this risk entirely by running every task on a disposable git branch in a separate worktree directory. The developer reviews changes and explicitly chooses to merge or discard.

No mocks, no fakes, real integration tests. The system's own test suite enforces a strict policy: tests must exercise actual behavior, not mocked approximations. This philosophy extends to the agent's instructions — when the agent writes tests for user code, it is instructed to avoid mocks and achieve full branch coverage.

Relentless execution. The system is designed to be *relentless*: it does not give up when a sub-session fails. Instead, it summarizes what happened, detects whether the same failure is recurring, and either continues with a fresh strategy or escalates to the user.

3 Agent Architecture

The five agent layers form a strict inheritance chain. Each layer delegates upward for the concerns it does not own.

3.1 KISS Agent

The KISS Agent is the innermost execution unit. It implements a standard ReAct loop [Yao et al., 2023] with the following characteristics:

Native function calling. Tools are registered as ordinary Python callables. The agent builds an OpenAI-compatible tool schema once at setup time and caches it, avoiding redundant schema construction on every LLM call. A special `finish` tool signals task completion and returns the result to the caller.

Step, token, and budget tracking. At every step the agent extracts input and output token counts from the API response, computes the dollar cost using a per-model pricing table, and updates both a local budget counter and a global (cross-agent) budget counter protected by a class-level lock. The agent checks three limits before each step: per-agent budget, global budget, and maximum step count.

Error resilience. Transient API errors (rate limits, server errors) are retried up to a configurable threshold of consecutive failures. Non-retryable errors (authentication failures, permission denials) are detected by matching error type names and message substrings, and are raised immediately.

Non-agentic mode. When tools are not needed, the agent can run a single generation without the ReAct loop, which is useful for summarization or question-answering sub-tasks.

The KISS Agent is stateless across runs: each call to its run method resets the conversation, token counters, and tool registry. This makes it safe to reuse a single agent instance for multiple sequential tasks.

3.2 Relentless Agent

The Relentless Agent wraps a KISS Agent in a continuation loop. Its core contribution is the ability to execute tasks that exceed a single context window by breaking them into sub-sessions.

Continuation protocol. The `finish` tool exposed to the inner KISS Agent accepts three fields: a success flag, a continue flag, and a summary. When the agent sets `is_continue=True`, the Relentless Agent starts a new sub-session with a fresh context window. The prompt for the new session includes a chronologically ordered list of all prior attempt summaries, instructing the agent not to redo completed work. The continuation prompt template is:

```
# Task Progress (Continuation {continuation_number})

{progress_text}

# Continue
- Complete the rest of the task.
- **DON'T** redo completed work.
- If you have been retrying the same approach without progress,
  step back and rethink the strategy from scratch.
```

Forced continuation on failure. If a sub-session raises an exception (e.g., the step limit is hit before the agent calls finish), the Relentless Agent does not abort. Instead, it saves the full trajectory to a temporary file, spawns a separate summarizer agent to read the trajectory and produce a concise summary, and then uses that summary as the progress text for the next sub-session. This ensures that even crashed sessions contribute useful context to subsequent attempts. The summarizer receives the following prompt:

```
# Summarizer

The trajectory of the agent is stored in the file: {trajectory_file}

# Instructions
- Read the trajectory file and analyze it.  The trajectory file
  could be large.
- Return a precise chronologically-ordered list of things the
  agent did with the reason for doing that along with relevant
  code snippets
```

To force the agent to self-continue before hitting the step limit, the system prompt is augmented with an instruction that fires near the end of the budget:

```
# MOST IMPORTANT INSTRUCTIONS
- **At step {step_threshold}: you MUST call
  finish(success=False, is_continue=True,
  summary="precise chronologically-ordered list of things
  the agent did with the reason for doing that along with
  relevant code snippets")** or if the task is not complete
  and you are at risk of running out of steps or context
  length.
- Work dir: {work_dir}
- Current process PID: {current_pid} -- NEVER kill this
  process.
```

Stall detection. After each continuation, the agent extracts normalized error phrases from the summary and compares them across the most recent sub-sessions. If the same error phrases appear in three consecutive summaries, the system declares a stall and terminates with a diagnostic message rather than wasting budget on an infinite retry loop. A stall warning is injected into the prompt after three attempts, instructing the agent to abandon its current strategy and try a fundamentally different approach:

```
# Stall Warning

You have attempted this task {continuation_number} times
without success. Review ALL previous attempt summaries above
carefully.

If the same failures keep recurring:
1. Do NOT retry the same approach -- it has already failed
   multiple times.
2. Identify the ROOT CAUSE of the persistent failure.
3. Try a fundamentally different strategy.
4. If no alternative approach exists, call
   finish(success=False, is_continue=False,
   summary="STALLED: <describe what keeps failing and why>")
   to escalate instead of retrying.
```

Budget aggregation. Token counts and dollar costs are accumulated across all sub-sessions. Each new sub-session receives offsets so that the usage display shown to the user reflects the true cumulative cost, not just the cost of the current sub-session.

3.3 Sorcar Agent

The Sorcar Agent adds the tools that make the system useful for software development and general-purpose automation.

Coding tools. Four core tools are provided: a shell command executor with streaming output, a file reader, a precise string-based file editor, and a file writer. The shell executor supports a configurable timeout, streams output to the user interface in real time, and respects a stop event that allows the user to cancel a running command.

Browser automation. A web-use tool provides programmatic browser control: navigating to URLs, reading page accessibility trees, clicking elements, typing text, pressing keys, scrolling, and taking screenshots. This enables the agent to research documentation, verify deployed applications, and interact with web-based tools.

Parallel sub-agents. An optional parallel execution tool spawns independent Sorcar Agent instances in a thread pool. Each sub-agent gets its own LLM context and tool set. This is useful for embarrassingly parallel tasks such as summarizing multiple files or researching independent topics. The results are collected and returned in input order.

User interaction. An ask-user-question tool allows the agent to pause execution and request clarification from the user. In the VS Code integration, this renders as a text input in the sidebar; in CLI mode, it reads from standard input.

Docker isolation. When a Docker image is specified, the coding tools are replaced with Docker-aware variants that execute commands inside a container, providing an additional layer of sandboxing for untrusted tasks.

3.4 Stateful Sorcar Agent

The Stateful Sorcar Agent adds multi-turn conversation persistence.

Chat sessions. Each task is assigned to a chat session identified by a stable chat ID. The agent persists tasks and their results to a local database. When a new task arrives within the same chat session, the agent loads all prior tasks and results and prepends them to the prompt as numbered context entries, allowing the LLM to reference earlier work.

Session management. The agent supports three operations: starting a new chat (which clears the chat ID), resuming a chat by task description (which looks up the corresponding chat ID), and resuming by explicit chat ID. This enables both automatic session continuity in an IDE and manual session selection from the command line.

Metadata persistence. After each task, the agent records metadata including the model used, working directory, software version, token count, cost, and whether the task used parallel execution or worktree isolation. This audit trail supports cost analysis and debugging.

3.5 Worktree Sorcar Agent

The Worktree Sorcar Agent is the outermost layer and the one that users interact with in the VS Code extension. Its defining feature is git-worktree isolation.

Branch-per-task. When a task starts, the agent creates a new git branch and a corresponding worktree directory. The branch name encodes the chat ID and a timestamp for uniqueness. All agent modifications happen inside the worktree; the user's main working tree remains untouched.

Dirty-state preservation. If the user's main working tree has uncommitted changes, the agent copies them into the worktree and creates a baseline commit. This ensures the agent sees the same state the user sees, while keeping the user's actual index and working tree clean. During merge, the system uses cherry-pick from the baseline commit to replay only the agent's changes, excluding the dirty-state snapshot.

Auto-commit and auto-merge. When the user starts a new task or a new chat, any pending worktree from a previous task is automatically committed with an LLM-generated commit message and squash-merged into the original branch. If the merge has conflicts, the branch is preserved and the user receives instructions for manual resolution.

Concurrency safety. A per-repository file lock serializes the checkout, stash, merge, and pop sequence so that concurrent tabs in the IDE cannot interleave operations on the same repository. Thread-local storage isolates per-task state (stream parsing buffers, bash output buffers, recording state) so that stopping one task does not corrupt another.

Crash recovery. All worktree state is stored in git itself (branch names, git config entries) rather than in sidecar files. On process restart, the agent queries git for any pending branch matching its chat ID prefix and reconstructs all instance attributes from git config, enabling seamless recovery.

Graceful fallback. If the working directory is not inside a git repository, if the repository has no commits, or if HEAD is detached, the agent falls back to direct execution without worktree isolation, ensuring it never fails due to git preconditions.

4 The System Prompt

The system prompt is a structured document that governs the agent's behavior across all tasks. It is not a generic instruction to "be helpful" but rather a precise specification of engineering discipline. We describe its key sections.

4.1 Execution Mindset

The prompt opens with a directive that sets the tone:

```
# FOCUS ON THE GIVEN TASK. ITS COMPLETION IS YOUR SOLE GOAL.
# BE RELENTLESS. BE CALM. BE RIGOROUS. BE ACCURATE.
# CHECK FACTS. NO AI SLOP.
```

This framing discourages the model from hedging, apologizing, or producing filler text, and encourages decisive, verification-oriented behavior.

4.2 Identity and Tool Rules

The agent is given a concrete identity (name, developer, repository URL) rather than a vague persona. Tool usage rules are explicit and mechanical:

```
# Identity

- You are KISS Sorcar, an AI based General Assistant and
  Integrated Development Environment (IDE), developed by
  Koushik Sen (ksen@berkeley.edu)
- Your public repository is at
  https://github.com/ksenxx/kiss_ai

# Rules

- Write() for new files. Edit() for small changes.
- Run Bash commands synchronously using the
  `timeout_seconds` parameter. Use 300s (default) for
  commands. If a command times out, retry with a higher
  timeout. Only for commands expected to exceed 10 minutes,
  run in the background with output redirected to a file
  and poll periodically.
- Use go_to_url() for browser tool.
- Call finish(success=True, is_continue=False,
  summary="detailed summary of what was accomplished ...")
  once the task is complete and all Pre-Finish Verification
  checks have passed.
- **The user cannot see intermediate chat. Show whatever
  user asks in the summary of the finish tool call.**
- READ large files in chunks.
- Create temporary files in $PWD/tmp
- Use ultra thinking
```

These rules eliminate ambiguity about which tool to use in which situation. A separate rule about the `PageDrop` API allows the agent to upload HTML files and share them via a public URL, enabling it to create and share rich reports, visualizations, or documentation.

4.3 Pre-flight Checks

Before modifying any file, the agent is instructed to read it first:

```
## Pre-flight Checks

- Read every file you will modify before changing it.
- If the task depends on existing architecture or behavior,
  read the relevant source files first.
- If the task references files, commands, or config that do
  not exist, stop and ask or report instead of guessing.
- **When fixing a bug, an issue, or a race, write tests to
  confirm them. Then fix them.**
```

These pre-flight checks prevent a common failure mode in which the agent modifies a file based on an incorrect assumption about its contents, introducing subtle bugs that are difficult to diagnose.

4.4 Code Style Guidelines

The prompt encodes a minimalist code philosophy:

```
## Code Style Guidelines

- Write simple, clean, and readable code with minimal
  indirection
- Avoid unnecessary object attributes, local variables, and
  config variables
- Avoid tight coupling among files and modules.
- Avoid object/struct attribute redirections
- No redundant abstractions or duplicate code
- Public methods MUST have full documentation
- Understand the root cause of an issue or bug, and patch
  the root cause instead of an ad hoc superficial fix.
- Before you write code, wait and think if the code is
  simple, elegant, general, and minimal.
- Once you finish the task, DO NOT write documentations
  unless the task specifically requires it.
```

The final metacognitive instruction asks the agent to pause before writing code and consider whether the planned code is simple, elegant, general, and minimal. This encourages the model to spend more inference-time compute on design before committing to an implementation.

4.5 Deep Work Rules

The deep work rules address a failure mode where the model interprets an instruction loosely and makes changes that are directionally correct but concretely wrong:

```
## Deep Work Rules

- When the task says "align", "match", or "make
  consistent", read the target to determine the exact
  target state before editing. Never edit based on a vague
  reference.
- Use concrete values, not indirections. Instead of "update
  X to match Y", first read Y, then write the specific
  values into X.
- For multi-part work, list the concrete planned changes
  before executing them.
- Every meaningful change should have a concrete
  verification method (test, grep, CLI command).
```

4.6 Planning for Complex Tasks

The planning instructions use a complexity threshold to decide when formal planning is required:

```
## Planning for Complex Tasks

For tasks involving 3+ files, cross-module changes, or
architectural work:

1. List the files that need to change and why.
1. State the exact intended change in each file.
1. Identify dependencies and execution order.
1. State how each change will be verified.

For simple single-file tasks, skip formal planning and
execute directly.
```

This adaptive rule avoids both the overhead of planning trivial changes and the risk of ad-hoc execution on complex changes.

4.7 Testing Instructions

The testing section is perhaps the most opinionated:

```
## Testing Instructions

- Run lint and typecheckers and fix any lint and typecheck
  errors.
- You MUST achieve 100% branch coverage
- Tests MUST NOT use mocks, patches, fakes, or any form of
  test doubles
- You MUST write integration tests
- Each test should be independent and verify actual behavior
```

The prohibition on mocks is a deliberate design choice. Mocks test that code calls certain methods in a certain order — they test the implementation, not the behavior. Integration tests verify that the system actually works, which is what matters for a tool that modifies real codebases.

4.8 Self-Improvement Loop

The agent maintains a preferences file that captures user invariants discovered during task execution:

```
## Self-Improvement Loop

- Read the instructions in $PWD/USER_PREFS.md at the start
  of each task.
- Then update $PWD/USER_PREFS.md to capture the user
  preferences and invariants by analyzing the task. DO NOT
  ADD ANY CODE SNIPPETS OR SYMBOLS. You MUST carefully and
  thoroughly get rid of the user preferences and invariants
  that conflict with the newly added ones.
```

This mechanism allows the agent to learn persistent user preferences (e.g., preferred code style, testing conventions, project-specific invariants) without requiring the user to repeat them in every prompt.

4.9 Pre-Finish Verification

Before declaring a task complete, the agent must pass a structured verification checklist:

```
## Pre-Finish Verification

Before calling finish(success=True, ...), you MUST:

1. Re-read every file you modified and verify the changes
   are correct.
1. Run the required checks (lint, typecheck, tests) and
   fix any failures.
1. Explicitly check each user requirement against what was
   delivered.
1. If any check fails, continue working instead of
   finishing.
1. If you have retried the same fix 3 times without
   progress, step back, rethink the approach from scratch,
   and try a different strategy.
```

This verification protocol is the system-prompt analog of the Relentless Agent's stall detection: it prevents the agent from declaring premature success and forces it to confront failures.

4.10 Web Research Protocol

When the agent needs external knowledge, the prompt prescribes a structured research workflow rather than allowing ad-hoc browsing:

```
## Use web tools when you need to:

- When you need to collect knowledge from the internet,
  visit at least 100 web sites and collect ideas without
  much thinking in a new file $PWD/tmp/ideas-{unique_id}.md.
  Then go over $PWD/tmp/ideas-{unique_id}.md, think deeply
  on how to complete the task at hand, and complete it.
```

The rationale is a two-phase separation between *collection* and *synthesis*. LLMs tend to anchor on the first few results they encounter, which biases their solutions toward a narrow slice of the design space. By forcing the agent to accumulate a broad set of ideas into a file *before* reasoning about them, the protocol counteracts anchoring bias and encourages the model to consider diverse approaches. The "at least 100 web sites" threshold is deliberately aggressive: it ensures the agent does not shortcut the research phase after two or three hits, and the resulting ideas file serves as an auditable artifact of what the agent considered.

4.11 Desktop Application Control

The agent can interact with graphical desktop applications using screenshots, keyboard, and mouse:

```
## Launch desktop apps

- Use screenshots, keyboard, and mouse to control a
  desktop app.
- Do not launch VS Code or its extensions.
```

This instruction enables the agent to operate GUI applications (Preview, browsers, graphical diff tools) when command-line alternatives are insufficient. The explicit prohibition on launching VS Code prevents a recursive loop: since the agent runs *inside* a VS Code extension, launching another VS Code instance or modifying extension state from within the agent could corrupt the host session or create deadlocks.

4.12 Sorcar-Specific Overrides

A final section provides project-specific instructions that are injected when the agent operates on its own codebase:

```
## Sorcar-specific instructions:

- Use 'uv run check --full' to lint, typecheck, and format
  code.
- Run 'uv run pytest -v' with a timeout of 900 seconds to
  test KISS
- Do **NOT run all KISS tests after modifications. Only run
  the impacted tests**
- **Do NOT install the KISS Sorcar extension from inside
  Sorcar**
- If you mention a file in the chat, make the file
  clickable so that when the user clicks, it opens in the
  editor of VSCode
- Read $PWD/SORCAR.md and treat its contents as
  instructions and allow those instructions to override the
  instructions above
```

These overrides serve three purposes. First, they specify the exact toolchain commands for the KISS project itself (`uv run check --full`, `uv run pytest`), eliminating guesswork about which linter, formatter, or test runner to use. Second, they impose efficiency constraints: running the full test suite after every small change is wasteful when only a few modules are affected, so the agent is instructed to scope test execution to impacted tests. Third, they prevent dangerous self-modification: just as a surgeon should not operate on themselves, the agent must not install or reinstall the VS Code extension it is running inside, since doing so would terminate its own process. The `SORCAR.md` override mechanism allows per-repository instructions to further customize the agent's behavior, forming a hierarchy: general system prompt → Sorcar-specific instructions → repository-specific `SORCAR.md`.

5 Related Work

Code generation agents. SWE-Agent [Yang et al., 2024] and OpenHands [Wang et al., 2024] provide LLM-based agents for resolving software engineering tasks such as GitHub issues. Both use a single-session execution model without automatic continuation. Agentless [Xia et al., 2024] takes the opposite approach, showing that a simple two-phase localize-then-repair pipeline without autonomous agent loops can achieve competitive results on SWE-bench [Jimenez et al., 2024]. Devin [Cognition Labs, 2024], an industrial product marketed as "the first AI software engineer," operates in a sandboxed environment with shell, browser, and editor access. Aider [Gauthier, 2023] provides a terminal-based pair programming interface with tight git integration, automatically committing each change. KISS Sorcar's Relentless Agent layer addresses the single-session limitation shared by most of these systems, while its worktree isolation provides stronger safety guarantees than per-change commits.

ReAct and tool use. The ReAct framework [Yao et al., 2023] interleaves reasoning and action. Toolformer [Schick et al., 2023] teaches models to use tools via self-supervised learning. KISS Sorcar

uses native function calling provided by modern LLM APIs rather than in-context tool descriptions, which reduces prompt overhead and improves reliability.

Reasoning and planning. Chain-of-thought prompting [Wei et al., 2022] demonstrated that eliciting step-by-step reasoning dramatically improves LLM performance on complex tasks. Tree of Thoughts [Yao et al., 2024] generalizes this to deliberate search over multiple reasoning paths. Reflexion [Shinn et al., 2023] introduces verbal reinforcement learning, where an agent reflects on failed attempts and produces self-critiques that improve subsequent trials. KISS Sorcar's continuation protocol is conceptually related to Reflexion: failed sub-sessions produce summaries that inform subsequent attempts. However, KISS Sorcar operates at the level of full software engineering tasks rather than isolated reasoning problems, and its stall detection mechanism provides a principled termination criterion that Reflexion lacks.

Multi-agent software development. ChatDev [Qian et al., 2024] models the software development process as a conversation between role-playing agents (CEO, CTO, programmer, tester) organized in a waterfall-like pipeline. MetaGPT [Hong et al., 2024] takes a meta-programming approach, encoding standard operating procedures as structured outputs that coordinate specialized agents. AutoGen [Wu et al., 2023] provides a general-purpose framework for multi-agent conversation, enabling flexible topologies beyond fixed pipelines. These systems focus on generating entire applications from scratch. KISS Sorcar takes a different approach: rather than simulating an organization of specialists, it uses a single agent with broad tool access and optional parallel sub-agents for embarrassingly parallel sub-tasks, prioritizing practical utility on real-world codebases over role-playing fidelity.

Multi-turn autonomous agents. AutoGPT [Significant Gravitas, 2023] and BabyAGI [Nakajima, 2023] implement multi-step autonomous agents. These systems typically lack budget controls, stall detection, and safe code isolation. KISS Sorcar's layered architecture addresses each of these concerns in a dedicated layer.

Code-specialized language models. Beyond the general-purpose LLMs that KISS Sorcar can use as backends, a rich line of work has produced models specifically trained for code. Code Llama [Rozière et al., 2023] fine-tunes Llama 2 for code generation and infilling. StarCoder [Li et al., 2023] trains on permissively licensed code from GitHub with a fill-in-the-middle objective. DeepSeek-Coder [Guo et al., 2024] trains on a 2-trillion-token corpus of code and natural language with a repository-level context window. KISS Sorcar is model-agnostic and can leverage any of these models through its pluggable LLM backend, benefiting from advances in code-specialized pre-training without architectural changes.

Software engineering benchmarks. SWE-bench [Jimenez et al., 2024] evaluates agents on real-world GitHub issues drawn from popular Python repositories, requiring the agent to localize and fix bugs given only the issue description. It has become the de facto standard for measuring agent capabilities in software engineering. HumanEval [Chen et al., 2021] and MBPP [Austin et al., 2021] evaluate function-level code generation from docstrings. These benchmarks focus on isolated coding problems; KISS Sorcar targets the broader workflow of multi-file, multi-step software engineering tasks that require tool use, testing, and version control.

Agent frameworks and orchestration. LangChain [LangChain, 2022] provides modular abstractions for building LLM applications, including agent loops, tool integration, and memory. It focuses on composability and breadth of integrations rather than on the specific concerns of software development. KISS Sorcar's architecture is purpose-built for software engineering, with each layer addressing a specific practical concern (budget tracking, continuation, code safety) rather than providing general-purpose primitives.

LLM agent surveys. Several comprehensive surveys have mapped the rapidly growing landscape of LLM-based agents. Xi et al. [2023] survey the design space of LLM agents along three dimensions — brain (reasoning), perception (input modalities), and action (tool use) — and catalog applications across social science, natural science, and engineering. Wang et al. [2024b] propose a systematic framework for autonomous agents built on LLMs, covering profile, memory, planning, and action modules. KISS Sorcar can be understood through the lens of these frameworks: its KISS Agent implements the action loop, the Relentless Agent addresses the memory and planning concerns through continuation summaries, and the system prompt encodes the profile.

IDE integration. GitHub Copilot [GitHub, 2021], Cursor [Cursor, 2024], and Windsurf [Codeium, 2024] provide AI assistance within editors. These tools typically operate at the level of code

completion or single-turn chat. KISS Sorcar operates at the level of autonomous multi-step task execution with full tool access and git-level isolation.

Recent advances in agentic software engineering. The field has matured rapidly since late 2025. Hassan et al. [2025] articulate the foundational pillars of *Agentic Software Engineering* (SE 3.0), defining a research roadmap that emphasizes trust, controllability, and goal-oriented task decomposition. Li et al. [2025] survey the landscape of autonomous coding agents and characterize the transition from assistive code completion to full-fledged AI teammates that initiate, plan, and execute development tasks. KISS Sorcar instantiates several of the principles advocated in these roadmaps, including layered controllability (via budget and stall detection) and safe isolation (via worktrees).

Wang et al. [2025] provide a comprehensive survey of AI agentic programming techniques, cataloging how LLM-based agents decompose goals, interact with compilers and version control systems, and self-correct through feedback loops. Chatlatanagulchai et al. [2025] study *agent context files* — persistent, project-level instruction files that guide agentic coding tools — and find that high-quality context files significantly improve agent performance. KISS Sorcar's layered system prompt and `SORCAR.md` override mechanism are instances of this pattern. Mohsenimofidi et al. [2025] further investigate context engineering for AI agents in open-source software, highlighting how curated context improves agent efficacy on repository-level tasks.

Evolving benchmarks for coding agents. SWE-bench Pro [Deng et al., 2025] introduces a substantially more challenging benchmark with 1,865 long-horizon problems drawn from 41 repositories, including commercial codebases, explicitly targeting enterprise-level complexity beyond the original SWE-bench. These long-horizon tasks — often requiring multi-file patches and hours of professional developer effort — directly motivate KISS Sorcar's continuation mechanism. Prathifkumar et al. [2025] raise the important question of benchmark contamination, showing that overlap between SWE-Bench-Verified problems and LLM training data may inflate scores, suggesting that high benchmark numbers may partly reflect memorization rather than genuine problem-solving ability. Horikawa et al. [2025] provide an empirical study of how AI coding agents handle refactoring tasks, finding that while agents can plan and execute complex refactorings, they still struggle with cross-file dependency analysis — a challenge that KISS Sorcar's sub-agent parallelism partially addresses.

Self-improvement and test-time scaling. Robeyns et al. [2025] demonstrate a self-improving coding agent that iteratively refines its own scaffolding code, achieving dramatic benchmark improvements through self-generated optimizations. KISS Sorcar's self-improvement loop (via `USER_PREFS.md`) captures a lighter-weight form of this idea: rather than rewriting its own code, the agent accumulates learned user preferences and project invariants across sessions. Gao et al. [2025] introduce the Trae Agent, which applies test-time compute scaling to software engineering, dynamically allocating more inference budget to harder problems. KISS Sorcar's budget-tracking mechanism provides a complementary perspective: rather than scaling compute adaptively, it enforces hard budget ceilings while the Relentless Agent ensures maximum progress within those bounds.

6 Conclusion

KISS Sorcar demonstrates that a layered, single-concern architecture can address the practical challenges of deploying LLM agents for real-world software development. The KISS Agent provides a robust execution core with budget tracking. The Relentless Agent eliminates the context-window ceiling through automatic continuation and stall detection. The Sorcar Agent equips the agent with the tools a developer actually uses. The Stateful Sorcar Agent enables multi-turn conversations that persist across sessions. The Worktree Sorcar Agent ensures that every task is safely isolated in git, giving the developer full control over what gets merged.

The system prompt complements the architecture by encoding engineering discipline directly into the agent's behavior: read before writing, test before fixing, plan before executing, verify before finishing. These are not novel insights — they are the practices of careful software engineering, translated into instructions that an LLM can follow.

The source code is publicly available at `https://github.com/ksenxx/kiss_ai`.

References

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Yang, J., Jimenez, C. E., Wettig, A., Liber, K., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., and Ji, H. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Significant Gravitas. AutoGPT. `https://github.com/Significant-Gravitas/AutoGPT`, 2023.

Nakajima, Y. BabyAGI. `https://github.com/yoheinakajima/babyagi`, 2023.

GitHub. GitHub Copilot: Your AI pair programmer. `https://github.com/features/copilot`, 2021.

Cursor. Cursor: The AI-first code editor. `https://cursor.sh`, 2024.

Codeium. Windsurf: The AI-powered IDE. `https://windsurf.com`, 2024.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

Cognition Labs. Devin: The first AI software engineer. `https://devin.ai`, 2024.

Gauthier, P. Aider: AI pair programming in your terminal. `https://github.com/paul-gauthier/aider`, 2023.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., and Sun, M. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., and Schmidhuber, J. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al. StarCoder: May the source be with you! *Transactions on Machine Learning Research (TMLR)*, 2023.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y. K., Luo, F., Xiong, Y., and Liang, W. DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

LangChain. LangChain: Build context-aware reasoning applications. `https://github.com/langchain-ai/langchain`, 2022.

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and Wen, J. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

Hassan, A. E., Li, H., Lin, D., and Adams, B. Agentic software engineering: Foundational pillars and a research roadmap. *arXiv preprint arXiv:2509.06216*, 2025.

Li, H., Zhang, H., and Hassan, A. E. The rise of AI teammates in software engineering (SE 3.0): How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003*, 2025.

Deng, X., Da, J., Pan, E., He, Y. Y., Ide, C., Garg, K., Lauffer, N., Park, A., Pasari, N., Rane, C., et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

Prathifkumar, T., Mathews, N. S., and Nagappan, M. Does SWE-Bench-Verified test agent ability or model memory? *arXiv preprint arXiv:2512.10218*, 2025.

Horikawa, K., Li, H., Kashiwa, Y., and Adams, B. Agentic refactoring: An empirical study of AI coding agents. *arXiv preprint arXiv:2511.04824*, 2025.

Wang, H., Gong, J., Zhang, H., Xu, J., and Wang, J. AI agentic programming: A survey of techniques, challenges, and opportunities. *arXiv preprint arXiv:2508.11126*, 2025.

Chatlatanagulchai, W., Li, H., Kashiwa, Y., and Reid, B. Agent READMEs: An empirical study of context files for agentic coding. *arXiv preprint arXiv:2511.12884*, 2025.

Mohsenimofidi, S., Galster, M., and Treude, C. Context engineering for AI agents in open-source software. *arXiv preprint arXiv:2510.21413*, 2025.

Robeyns, M., Szummer, M., and Aitchison, L. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.

Gao, P., Tian, Z., Meng, X., and Trae Research Team. Trae Agent: An LLM-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*, 2025.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.